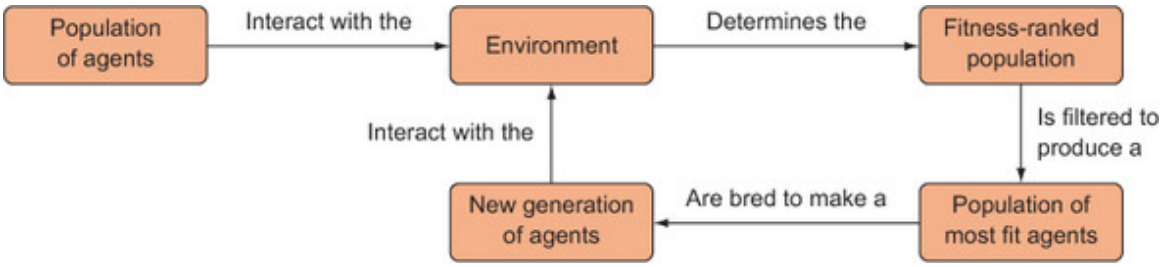
Instead of creating one agent and improving it, we can instead learn from Charles Darwin and use evolution by (un)natural selection. We could spawn multiple different agents with different parameters (weights), observe which ones did the best, and "breed" the best agents such that the descendants could inherit their parents' desirable traits—just like in natural selection. We could emulate biological evolution using algorithms. We wouldn't need to struggle to tune hyperparameters and wait for multiple epochs to see if an agent is learning "correctly." We could just pick the agents that are already performing better (figure 6.2).

Figure 6.2. Evolutionary algorithms are different from gradient descent-based optimization techniques. With evolutionary strategies, we generate agents and pass the most favorable weights down to the subsequent agents.



This class of algorithms does not require an individual agent to learn. It does not rely on gradient descent and is aptly called a *gradient-free algorithm*. But just because individual agents are not being nudged toward some objective directly does not mean that we are relying on pure chance. The renowned evolutionary biologist Richard Dawkins once said, "Natural selection is anything but random." Similarly, in our quest to build, or more accurately *discover*, the best agent, we will not be relying on pure chance. We will be selecting for the fittest amongst a population with a variance in traits.

6.2. Reinforcement learning with evolution strategies

In this section we'll talk about how fitness plays into evolution strategies, and we'll briefly cover the task of selecting the fittest agents. Next, we'll work on how to recombine those agents into new agents and show what happens when we introduce mutations. This evolution is a multiple-generation process, so we'll discuss that and recap the full training loop.